

COMPLETE TUTORIAL

Building an Exam Study Website for Your Child

A complete step-by-step guide to creating an interactive, mobile-friendly study site with flashcards and online tests — using Visual Studio Code, GitHub Pages, and Jotform.

Based on a real-world build for a Grade 7 Afrikaans learner at Laerskool Jeffreysbaai. No prior coding experience required — just willingness to follow instructions carefully.

Table of Contents

1. What You Will Build & What You Need
2. Part 1 — Software & Accounts You Must Install
3. Part 2 — Capture the Source Material (Photos of Textbook)
4. Part 3 — Project Setup in Visual Studio Code
5. Part 4 — Build the HTML Skeleton (Tabbed Subject Layout)
6. Part 5 — Transcribing Textbook Photos into Study Plans
7. Part 6 — Designing Flashcards (The Card-Flip Pattern)
8. Part 7 — Creating Online Tests with Jotform
9. Part 8 — Mobile-Friendly Styling for Android Phones
10. Part 9 — Adding Collapsible Sections & Exam Dates
11. Part 10 — Publishing to GitHub Pages (Free Hosting)
12. Part 11 — Updating the Site After Launch
13. Part 12 — Troubleshooting Common Problems
14. Appendix — Reusable Code Templates & Checklists

1. What You Will Build & What You Need

The End Result

A single-page website hosted free on GitHub Pages where your child can:

- Tap a tab for each school subject
- See a study plan with exam date and topic breakdown
- Flip through interactive flashcards with a progress bar
- Click a button to take an online test (auto-marked via Jotform)
- Browse comfortably on an Android phone

See the live reference site: <https://flomaticauto.github.io/ian-eksamen-plan/>

Skill Level Required

None. If you can follow instructions and copy-paste text into the right places, you can build this. The hardest part is patience.

How Long It Takes

Stage	Time
Installing software (Part 1)	30–45 minutes
Photographing 1 subject's textbook pages	10 minutes per subject
Building the first subject tab	1–2 hours
Each additional subject (after the first)	30–45 minutes
Creating a Jotform test	20 minutes per subject
Publishing to GitHub Pages	15 minutes (one-time)

What You Will Need

- A Windows laptop or desktop (Mac works too — paths differ slightly)
- A smartphone to photograph textbook pages
- An email address (to register for GitHub and Jotform)
- About 4–6 hours total for a full subject build-out

Why this stack? Visual Studio Code is free and the most widely used code editor in the world. GitHub Pages gives you free hosting forever with a public web address. Jotform's free tier lets you build proper auto-marked tests without learning to code a backend. Together they cost ZERO rand per month.

2. Part 1 — Software & Accounts You Must Install

1.1 Install Visual Studio Code (VS Code)

1. Go to <https://code.visualstudio.com/>
2. Click the big blue "Download for Windows" button
3. Run the downloaded `.exe` file
4. Accept all default options during installation
5. When asked, tick "Add 'Open with Code' action to Windows Explorer file context menu" — this is very useful

1.2 Install Git (version control)

Git is the tool that pushes your files to GitHub.

1. Go to <https://git-scm.com/download/win>
2. The download starts automatically
3. Run the installer; accept all defaults
4. When prompted for default editor, choose **"Use Visual Studio Code as Git's default editor"**

Open Command Prompt (press Windows key, type "cmd", Enter) and check:

```
git --version
```

You should see something like `git version 2.52.0.windows.1`.

1.3 Create a GitHub Account

1. Go to <https://github.com/signup>
2. Sign up with an email — pick a username you don't mind being public (it will appear in your site's URL)
3. Verify the email
4. Skip all the optional onboarding steps

Tip: Your future site URL will look like <https://YOUR-USERNAME.github.io/PROJECT-NAME/>. Choose a username that looks neat to share.

1.4 Install the GitHub CLI (Command Line Interface)

This tool makes pushing code to GitHub far easier than the manual method.

1. Go to `https://cli.github.com/`
2. Click "Download for Windows"
3. Run the installer, accept defaults
4. Open Command Prompt and run:

```
gh auth login
```

Follow the prompts: choose **GitHub.com** → **HTTPS** → **Login with a web browser**. A one-time code appears; copy it. Your browser opens — paste the code and approve.

1.5 Create a Jotform Account

1. Go to `https://www.jotform.com/signup`
2. Sign up free — the free tier allows up to 5 forms with 100 submissions per month, more than enough
3. Verify your email

End of Part 1 checklist:

- ☐ VS Code installed and opens
- ☐ Git installed (`git --version` works)
- ☐ GitHub account created
- ☐ GitHub CLI installed and logged in (`gh auth status` shows "Logged in")
- ☐ Jotform account created

3. Part 2 — Capture the Source Material (Photos of Textbook)

Before you write any code, you need clear photos of every page from your child's textbook that you want included.

Photography Best Practice

- **Lighting:** Take photos near a window during the day. Avoid direct sun glare on the page.
- **Angle:** Hold the phone directly above the page, not at an angle. The text must be straight.
- **Focus:** Tap the screen on the text before pressing the shutter so the camera focuses correctly.
- **One page at a time:** Don't try to fit two pages in one photo. Resolution suffers.
- **Format:** Save as JPEG — smaller file size than PNG and good enough quality for text.

Organising the Photos on Your Computer

Create a clean folder structure. For the reference build, the structure was:

```
OneDrive\Resources\Ian\Kwartaal 1\  
├─ Wiskunde\  
│   ├─ page-1.jpeg  
│   ├─ page-2.jpeg  
│   └─ ...  
├─ Afrikaans\  
│   └─ ...  
├─ EBW\  
│   └─ ...  
└─ (one folder per subject)
```

Tip: WhatsApp-to-self is a fast way to move photos from phone to PC. Send the photos to yourself on WhatsApp Web, then save the images to your subject folders.

4. Part 3 — Project Setup in Visual Studio Code

3.1 Create a Project Folder

1. Open File Explorer
2. Go to `C:\Users\YOUR-USERNAME\`
3. Right-click → New Folder
4. Name it something like `eksamen-plan` or your child's name (e.g. `ian-eksamen-plan`)

3.2 Open the Folder in VS Code

1. Right-click on the folder → "Open with Code" (this option appeared because you ticked it during VS Code install)
2. VS Code opens with the folder visible on the left sidebar

3.3 Initialise Git in the Folder

In VS Code, open the integrated terminal: **View** → **Terminal** (or press `Ctrl+``).

Type these commands one at a time, pressing Enter after each:

```
git init
git checkout -b main
```

This creates a hidden `.git` folder that tracks every change you make.

3.4 Create the Main HTML File

1. In VS Code, click the "New File" icon in the sidebar (top of file list)
2. Name it exactly: `index.html`
3. The file opens — paste the starter HTML from **Appendix A.1** at the end of this guide

Why `index.html`? Web servers default to serving a file called `index.html` when someone visits a folder. GitHub Pages follows this same rule.

5. Part 4 — Build the HTML Skeleton (Tabbed Subject Layout)

The page has three core parts:

1. A **header** at the top with the title
2. A **tab bar** with one button per subject
3. One **content panel** per subject (only the active one is visible)

The Tab Switching Pattern

Every tab button has an `onclick` attribute calling a JavaScript function that:

1. Hides all content panels (removes the `aktief` class)
2. Removes the "active" highlight from all tab buttons
3. Shows the chosen panel
4. Highlights the clicked tab button
5. Scrolls back to the top of the page

The JavaScript Function

```
<script>
function showTab(e, id) {
  document.querySelectorAll('.tab-content').forEach(function(el) {
    el.classList.remove('active');
  });
  document.querySelectorAll('.tab-btn').forEach(function(el) {
    el.classList.remove('active');
  });
  document.getElementById(id).classList.add('active');
  e.currentTarget.classList.add('active');
  window.scrollTo({ top: 0, behavior: 'smooth' });
}
</script>
```

Important: Notice the function takes `(e, id)` as parameters. Older code uses the global `event` variable, but this is unreliable on some browsers. Pass the event explicitly from the button: `onclick="showTab(event, 'wiskunde1')"`.

The HTML for One Tab Button + Content Panel

```
<!-- The button -->
<button class="tab-btn active" onclick="showTab(event,'wiskunde1')">
  Wiskunde V1
</button>

<!-- The panel (shown when the button is clicked) -->
<div id="wiskunde1" class="tab-content active">
  <h2>Wiskunde – Vraestel 1</h2>
  <p>Exam date: ...</p>
  <!-- study plan goes here -->
</div>
```

Repeat this pattern for each subject. Only the **first** button and the **first** panel start with the `active` class.

6. Part 5 — Transcribing Textbook Photos into Study Plans

This is the most time-consuming step but the most valuable. You're converting the photos into structured digital text.

Option A — Manual Transcription

Open each photo, read it carefully, and type the content into `index.html` as a list. Best for accuracy on handwritten notes.

Option B — AI-Assisted Transcription

Use Claude (claude.ai), ChatGPT, or another AI with vision support. Upload the photo and ask:

```
Transcribe this textbook page exactly as written. Keep
the language (Afrikaans/English). Preserve section
numbers and bullet structure.
```

Review the output for errors — AI sometimes mis-reads handwriting or specialised terms.

The HTML Structure for a Study Plan

```
<div class="section">
  <h3>5. Inleiding tot Rekeningkunde</h3>
  <ul>
    <li><strong>5.1</strong> Rekeningkunde is...</li>
    <li><strong>5.2</strong> Belangrike terme...</li>
  </ul>
</div>
```

Tip: Don't try to copy the textbook word-for-word. Summarise. Your child reads this on a phone — long paragraphs are unreadable. Use bullet points, bold key terms, group related concepts.

7. Part 6 — Designing Flashcards (The Card-Flip Pattern)

Each flashcard has a **front** (the question) and a **back** (the answer). Tapping the card flips it over in 3D.

The HTML for One Card

```
<div class="flashcard" onclick="flipCard(this)">
  <div class="card-inner">
    <div class="card-front">
      <span class="card-number">1</span>
      <div class="question">What is a Lyn in art?</div>
      <div class="hint">Element of Visual Art</div>
    </div>
    <div class="card-back">
      <div class="answer">
        Lyn is the most basic element of art...
      </div>
      <div class="trick">💡 Memory hook here</div>
    </div>
  </div>
</div>
```

The CSS That Makes the Flip Work

```
.flashcard {
  height: 180px;
  perspective: 900px; /* enables 3D effect */
  cursor: pointer;
}
.card-inner {
  position: relative;
  width: 100%; height: 100%;
  transform-style: preserve-3d;
  transition: transform 0.5s;
}
.flashcard.flipped .card-inner {
  transform: rotateY(-180deg);
}
.card-front, .card-back {
  position: absolute; inset: 0;
  backface-visibility: hidden;
}
.card-back {
  transform: rotateY(180deg);
  background: white;
}
```

How the trick works: Both faces are stacked on top of each other. The back starts rotated 180°, so it's facing AWAY from you. When the card flips, both rotate together — the front spins to face away, the back rotates 180° to face you. `backface-visibility: hidden` means each face disappears when its back is turned, so you only see one at a time.

The JavaScript

```
var learned = new Set();
function flipCard(card) {
  var wasFlipped = card.classList.contains('flipped');
  card.classList.toggle('flipped');
  var id = card.querySelector('.card-number').textContent;
  if (!wasFlipped) {
    learned.add(id);
  } else {
    learned.delete(id);
  }
  updateProgress();
}
function updateProgress() {
  var total = 13;
  var pct = Math.round((learned.size / total) * 100);
  document.getElementById('progress-bar').style.width = pct + '%';
  document.getElementById('counter').textContent =
    learned.size + ' / ' + total + ' learned';
}
```

Important: Each subject needs its OWN `learned` Set and its OWN flip function (e.g. `flipKunsCard`, `flipEbwCard`). Otherwise progress on one subject overwrites the other.

How Many Cards Per Subject?

10–20 is the sweet spot. Less than 10 feels thin; more than 20 is overwhelming. Group them visually by "day" or "topic" with header dividers.

8. Part 7 — Creating Online Tests with Jotform

Jotform handles the test rendering, multiple-choice randomisation, auto-marking, and result emails. You just write the questions.

7.1 Start a New Form

1. Log into Jotform at <https://www.jotform.com/myforms/>
2. Click "Create Form" (top right)
3. Choose "Start from Scratch"
4. Pick "Classic Form" layout (one long page, easier for younger learners)

7.2 Add a Form Title

At the top, click "Form Title" and rename it to something like `EBW Kwartaal 1 Toets – Graad 7`.

7.3 Add Name & Date Fields

1. Click "Add Form Element" (left panel)
2. Drag a "Short Text" field — rename label to "Naam en Van"
3. Drag a "Date Picker" field — rename label to "Datum"

7.4 Add Multiple-Choice Questions

1. Drag a "Single Choice" element for each question
2. In the question text, type the question (e.g. "Wat is rekeningkunde?")
3. In the options list, put the CORRECT answer first, then the wrong ones
4. Open the field's "Properties" → "Options" tab
5. Enable "Randomize Options" so the correct answer's position changes

7.5 Enable Auto-Marking (Quiz Mode)

1. Click "Settings" in the top bar
2. Find the "Quiz" tab (sometimes under "Add-ons" or "Widgets" depending on Jotform version)
3. Mark each question as a quiz question and tick the correct answer
4. Set a point value (1 point each works fine for a 20-question test)
5. Enable "Show Score After Submission"

7.6 Add a Thank-You Message

1. Click "Settings" → "Thank You Page"
2. Choose "Show Custom Text"
3. Write: `Goeie werk! Jou punt is hierbo gewys. Probeer weer as jy minder as 80% gekry het.`

7.7 Get the Form URL

1. Click "Publish" in the top bar
2. Copy the "Link to Share" — it looks like `https://form.jotform.com/261404882092053`
3. This is what you'll put in your HTML test button

Adding the Test Button to Your HTML

```
<a href="https://form.jotform.com/261404882092053"
  target="_blank"
  class="test-button">
  ✎ Doen die EBW Toets
</a>
```

And the CSS:

```
.test-button {
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #059669, #047857);
  color: white;
  text-decoration: none;
  border-radius: 12px;
  padding: 14px 20px;
  font-weight: 700;
  margin-bottom: 16px;
}
```

Tip: The `target="_blank"` opens the test in a new tab, so your child doesn't lose their place on the study site.

9. Part 8 — Mobile-Friendly Styling for Android Phones

The site **MUST** work well on a phone — that's how most kids will use it.

The Critical Meta Tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Without this, mobile browsers will render the page as if it were 1000px wide and zoom out, making everything tiny.

Horizontally Scrollable Tab Bar

On mobile, fitting 11 tabs on one row is impossible. Make the tab bar scroll sideways:

```
.tab-nav {
  display: flex;
  flex-wrap: nowrap;           /* no wrapping to next row */
  overflow-x: auto;           /* enable horizontal scroll */
  -webkit-overflow-scrolling: touch; /* smooth on iOS */
  scrollbar-width: none;      /* hide scrollbar */
}
.tab-nav::-webkit-scrollbar { display: none; }
.tab-btn {
  flex-shrink: 0;             /* don't squish buttons */
  white-space: nowrap;         /* keep text on one line */
  min-height: 44px;           /* finger-friendly */
}
```

Why 44px? That's Google's and Apple's minimum recommended touch target size. Anything smaller and people mis-tap consistently.

Avoid `:hover` Styles on Mobile

On phones, `:hover` stays "stuck" after tap — looks broken. Use `:active` instead (fires only during the tap):

```
.tab-btn:active { /* not :hover */
  background: rgba(233,69,96,0.4);
}
```

Disable the Grey Tap Highlight

```
.tab-btn { -webkit-tap-highlight-color: transparent; }
```

Responsive Breakpoints

```
@media (min-width: 600px) {  
  /* styles for tablets and larger */  
  .tab-nav { flex-wrap: wrap; overflow-x: visible; }  
  .flashcard { height: 200px; }  
}
```


10. Part 9 — Adding Collapsible Sections & Exam Dates

The Collapse Pattern (Accordion)

Some sections (study plan, flashcards) take a lot of vertical space. Hide them behind a click-to-expand button.

```
<div class="collapse-section">
  <button class="collapse-head" onclick="toggleCollapse('panel1', this)">
    <span class="arrow">▶</span>
    <span>📅 Study Plan</span>
  </button>
  <div class="collapse-body" id="panel1">
    <!-- hidden content -->
  </div>
</div>
```

The CSS uses a `max-height` trick (you can't transition `height: auto`, but you can transition `max-height`):

```
.collapse-body {
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.4s ease;
}
.collapse-body.open {
  max-height: 4000px; /* large enough for any content */
}
.collapse-head.open .arrow {
  transform: rotate(90deg); /* points down when open */
}
.arrow { transition: transform 0.3s; }
```

The JavaScript:

```
function toggleCollapse(panelId, button) {
  document.getElementById(panelId).classList.toggle('open');
  button.classList.toggle('open');
}
```

Exam Date Headers

Above the study plan, show the exam date prominently:

```
<div class="exam-date">📅 Exam: Thursday, 04 June 2026</div>
```

```
.exam-date {  
  font-size: 0.82rem;  
  color: #e94560;      /* red – eye-catching */  
  font-weight: 700;  
  margin-bottom: 18px;  
}
```

Tip: Include the day of the week (e.g. "Thursday, 04 June"). "04 June" is abstract; "Thursday" is concrete. A 12-year-old responds better to the day name.

11. Part 10 — Publishing to GitHub Pages (Free Hosting)

This is where your local files become a public website with a real URL.

10.1 Commit Your Files Locally

In VS Code's terminal:

```
git add index.html
git commit -m "Initial study site"
```

10.2 Create the GitHub Repo & Push

Still in the terminal:

```
gh repo create my-eksamen-plan --public --source=. --remote=origin --push --description
"Grade 7 study site"
```

Replace `my-eksamen-plan` with whatever you want the URL to say. The command:

- Creates a public repo with that name on GitHub
- Sets it as your "remote"
- Pushes your files in one go

10.3 Enable GitHub Pages

1. Go to `https://github.com/YOUR-USERNAME/my-eksamen-plan/settings/pages`
2. Under "Branch", choose **main** from the dropdown
3. Leave the folder as **/ (root)**
4. Click **Save**
5. Wait 1–2 minutes

10.4 Visit Your Live Site

Your URL is now:

```
https://YOUR-USERNAME.github.io/my-eksamen-plan/
```

Refresh after a couple of minutes if it's not live immediately.

How GitHub Pages works: It's a free static-site host built into GitHub. Every time you push a change to the `main` branch, GitHub rebuilds your site automatically. No "deploy" step needed.

12. Part 11 — Updating the Site After Launch

Once a subject is built, updating it takes about a minute. The cycle is:

1. Open `index.html` in VS Code
2. Make your changes (add a flashcard, fix a typo, add a topic)
3. Save (`Ctrl+S`)
4. In the terminal, run:

```
git add index.html
git commit -m "Add flashcards for Wiskunde V2"
git push
```

Wait 30 seconds. Your live site updates automatically.

Previewing Locally Before Pushing

Before pushing, you can preview your changes:

1. In VS Code, install the extension "Live Server" (by Ritwick Dey)
2. Right-click `index.html` → "Open with Live Server"
3. Your default browser opens with the site running locally
4. Every time you save, the browser auto-refreshes

13. Part 12 — Troubleshooting Common Problems

Problem	Cause & Fix
Tab buttons all open the same panel	The <code>id</code> attribute on the panels must be unique. Check for typos in <code>onclick="showTab(event, 'X')"</code> — the <code>X</code> must match an <code>id="X"</code> on a panel.
Card flip animation is jerky	Check that <code>.card-inner</code> has <code>transform-style: preserve-3d</code> . Without it, browsers fall back to a flat flip that looks broken.
Tapping the page bounces back to top	An <code></code> link is being used somewhere as a button. Replace with <code><button></code> or use <code>href="javascript:void(0)"</code> .
<code>git push</code> fails with HTTP 403	You're authenticated as the wrong GitHub user. Run <code>gh auth status</code> and <code>gh auth switch --user YOUR-USERNAME</code> .
GitHub Pages site shows 404	Check Settings → Pages is set to "main" branch. Also wait 2–3 minutes after the first push — GitHub needs time to build.
Jotform test button doesn't open the form	Check the URL has <code>https://</code> at the start, no trailing space, and <code>target="_blank"</code> .
Collapse animation closes very slowly	You set <code>max-height: 99999px</code> . Lower to <code>4000px</code> — the animation interpolates the full range even if the content is shorter.
Mobile tab bar wraps into multiple ugly rows	Add <code>flex-wrap: nowrap; overflow-x: auto</code> to the tab nav. See Part 8.

14. Appendix — Reusable Code Templates & Checklists

A.1 — Starter `index.html`

Copy this into your empty `index.html` to start:

```
<!DOCTYPE html>
<html lang="af">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My Eksamen Plan</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      background: #f0f4f8;
      color: #1a1a2e;
    }
    header {
      background: linear-gradient(135deg, #1a1a2e, #0f3460);
      color: white;
      text-align: center;
      padding: 28px 16px;
    }
    .tab-nav {
      display: flex;
      overflow-x: auto;
      gap: 6px;
      padding: 10px;
      background: #16213e;
    }
    .tab-btn {
      flex-shrink: 0;
      background: rgba(255,255,255,0.1);
      color: white;
      border: none;
      border-radius: 22px;
      padding: 10px 16px;
      font-weight: 600;
      cursor: pointer;
      min-height: 44px;
      white-space: nowrap;
    }
    .tab-btn.active { background: #e94560; }
    .tab-content {
      display: none;
      padding: 20px;
      max-width: 900px;
      margin: 0 auto;
    }
    .tab-content.active { display: block; }
  </style>
</head>
<body>

<header>
  <h1>My Eksamen Plan</h1>
</header>
```



```

<div class="tab-nav">
  <button class="tab-btn active" onclick="showTab(event, 'subject1')">Wiskunde</button>
  <button class="tab-btn" onclick="showTab(event, 'subject2')">Afrikaans</button>
</div>

<div id="subject1" class="tab-content active">
  <h2>Wiskunde</h2>
  <p>Content here</p>
</div>

<div id="subject2" class="tab-content">
  <h2>Afrikaans</h2>
  <p>Content here</p>
</div>

<script>
function showTab(e, id) {
  document.querySelectorAll('.tab-content').forEach(function(el) {
    el.classList.remove('active');
  });
  document.querySelectorAll('.tab-btn').forEach(function(el) {
    el.classList.remove('active');
  });
  document.getElementById(id).classList.add('active');
  e.currentTarget.classList.add('active');
  window.scrollTo({ top: 0, behavior: 'smooth' });
}
</script>

</body>
</html>

```

A.2 — Pre-Launch Checklist

- ☐ VS Code, Git, GitHub CLI, Jotform — all installed and tested
- ☐ One subject completely built: study plan + flashcards + test button
- ☐ Site previewed locally with Live Server
- ☐ All flashcards flip correctly
- ☐ Jotform test opens, accepts answers, and shows score
- ☐ Page looks good on phone (test by visiting on your phone)
- ☐ Pushed to GitHub successfully
- ☐ GitHub Pages enabled (Settings → Pages)
- ☐ Live URL works in an incognito browser window
- ☐ Shared the URL with your child

A.3 — Per-Subject Build Checklist

For each new subject, repeat this routine:

- ☐ Photograph all relevant textbook pages
- ☐ Transcribe into a structured study plan
- ☐ Write 10–20 flashcards (question on front, answer on back, memory hook)
- ☐ Create a Jotform test (15–20 multiple-choice questions, auto-marked)
- ☐ Add the tab button to the tab bar
- ☐ Add the tab content panel with study plan, flashcards, and test button
- ☐ Add unique flip function for this subject (`flipXKaart` , learned set)
- ☐ Test locally with Live Server
- ☐ Commit and push
- ☐ Verify on live URL

A.4 — Useful Commands Cheat Sheet

Command	Purpose
<code>git status</code>	See what files have changed
<code>git add filename</code>	Stage a file for commit
<code>git add .</code>	Stage all changed files
<code>git commit -m "message"</code>	Save a snapshot with a description
<code>git push</code>	Upload commits to GitHub (deploys to Pages)
<code>git log --oneline</code>	See history of all your changes
<code>gh auth status</code>	Check which GitHub account is active
<code>gh repo view --web</code>	Open your repo in the browser

A.5 — Where to Get Help

- **HTML/CSS reference:** MDN — <https://developer.mozilla.org/>
- **Git problems:** Stack Overflow — search the exact error message
- **VS Code questions:** <https://code.visualstudio.com/docs>
- **Jotform support:** <https://www.jotform.com/help/>
- **GitHub Pages docs:** <https://docs.github.com/en/pages>

Final word: The first subject takes longest. By the third subject, you'll be flying through it in 30 minutes. The whole site for 11 subjects took about 12 hours spread over a few sessions. Your child will use this every exam season for years.

Built and documented from the live project at
<https://flomaticauto.github.io/ian-eksamen-plan/>